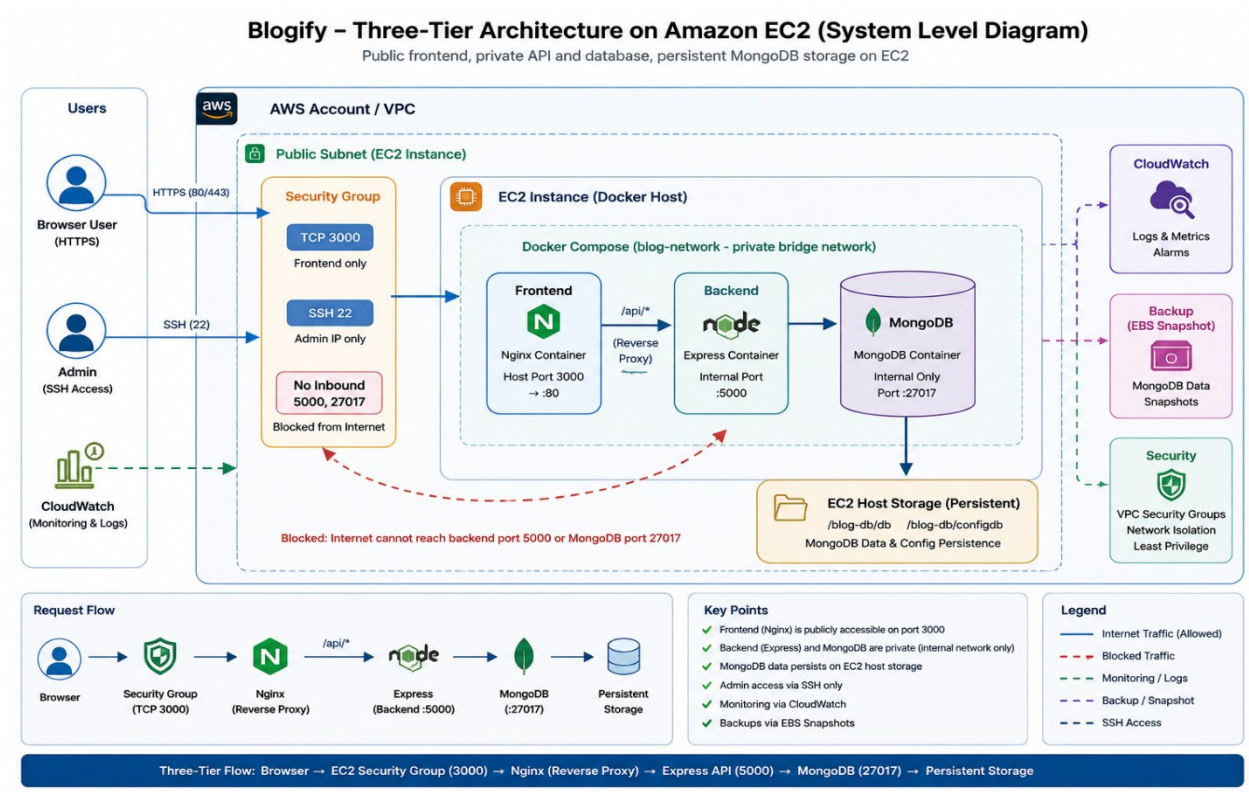
Phase 1 Exam Submission

1. As part of the exam, cloned an open-source **3-Tier project** that was not Dockerized.
2. Project Link: [Webblog-3tier]
3. A simple blogging platform where users can create, edit, comment on, search, like, and delete blog posts.
4. Tech Stack:-
   - Frontend: React, Tailwind CSS
   - Backend: node.js, express.js
   - Database: MongoDB
5. The application was not Dockerized, so implementing Docker containers and Docker Compose to containerize and run the complete 3-tier application on EC-2 instance.
6. System Design:-



Step 1: DockerFile for frontend & Backend.

Frontend:-

```
FROM node:latest
WORKDIR /app
COPY . .
RUN npm ci
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Backend:-

```
FROM node:26
WORKDIR /app
COPY . .
RUN npm ci
EXPOSE 5000
CMD ["node","index.js"]
```

- Frontend: 2.59GB & Backend: 1.85 GB

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
blog-backendend:v1	4398a9c767eb	1.85GB	454MB	
blog-frontend:v1	ed96b87cf37d	2.59GB	602MB	
mongo:7.0	b6421fd6d1c5	1.18GB	299MB	
mongo:latest	5211c51171f5	1.15GB	303MB	

We can clearly see here that images size is too big and we have to optimize it further. So, in this case, we can use lighter image possible. So in below updated Dockerfile, we just use alpine images which are not heavy & tagged with **v2** version.

Frontend:-

```
FROM node:alpine
WORKDIR /app
COPY . .
RUN npm ci
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Backend:-

```
FROM node:alpine
WORKDIR /app
COPY . .
RUN npm ci
EXPOSE 5000
CMD ["node","index.js"]
```

- Frontend: 1.06GB→60% Optimization
- Backend: 317MB→70% Optimization

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
blog-backendend:v1	4398a9c767eb	1.85GB	454MB	
blog-backendend:v2	ad7d5df08edf	317MB	74.7MB	
blog-frontend:v1	ed96b87cf37d	2.59GB	602MB	
blog-frontend:v2	78e21bbe3412	1.06GB	222MB	
mongo:7.0	b6421fd6d1c5	1.18GB	299MB	
mongo:latest	5211c51171f5	1.15GB	303MB	

Step 2: Use Multi-Stage Build to Optimize Image

(Now we are using multi-stage build concepts with lighter images)

Frontend:-

```
FROM dhi.io/node:26-dev AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine AS runner
WORKDIR /app
# COPY package.json package-lock.json ./
# COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Backend:

```
FROM node:alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:alpine AS runner
COPY . .
COPY --from=build /app/node_modules ./node_modules
EXPOSE 5000
CMD ["node","index.js"]
```

- Frontend v3: 1.06GB → 110MB (90% optimized)
- Backend v3: 317MB → 298MB (10% optimized)

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
blog-backend:v3	344ce9db47e4	298MB	68.6MB	
blog-backendend:v1	4398a9c767eb	1.85GB	454MB	
blog-backendend:v2	ad7d5df08edf	317MB	74.7MB	
blog-frontend:v1	ed96b87cf37d	2.59GB	602MB	
blog-frontend:v2	78e21bbe3412	1.06GB	222MB	
blog-frontend:v3	9c3ce8178cc3	110MB	30.6MB	
mongo:7.0	b6421fd6d1c5	1.18GB	299MB	
mongo:latest	5211c51171f5	1.15GB	303MB	

Step 3: Use Docker Hardened images to Optimize Image.

Frontend is already using dhi image in earlier dockerfile.
Now, using dhi images for backend & slim image for frontend.

Frontend:-

```
FROM dhi.io/node:26-dev AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:stable-alpine-slim AS runner
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Backend:-

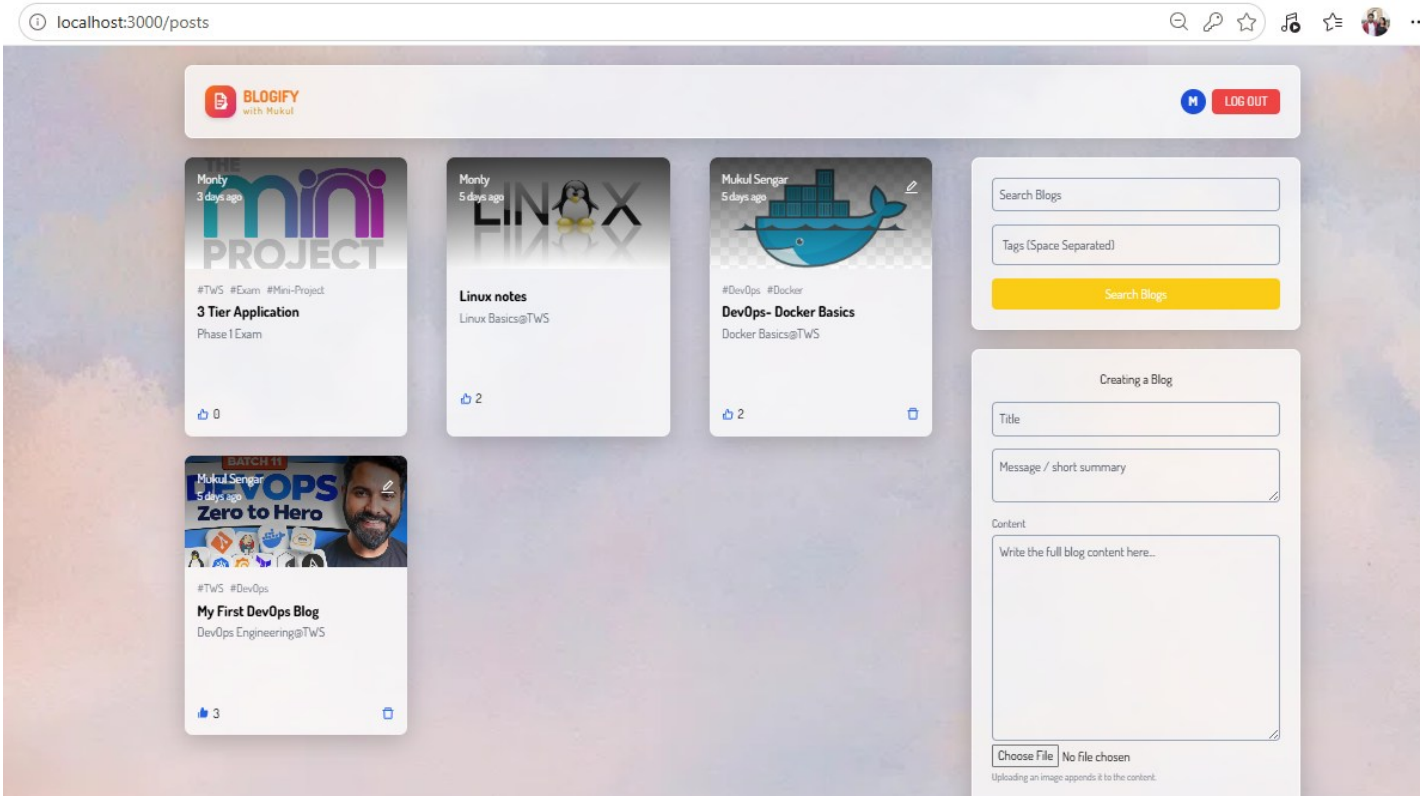
```
FROM dhi.io/node:26-dev AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM dhi.io/node:26 AS runner
WORKDIR /app
COPY . .
COPY --from=build /app/node_modules ./node_modules
EXPOSE 5000
CMD ["node","index.js"]
```

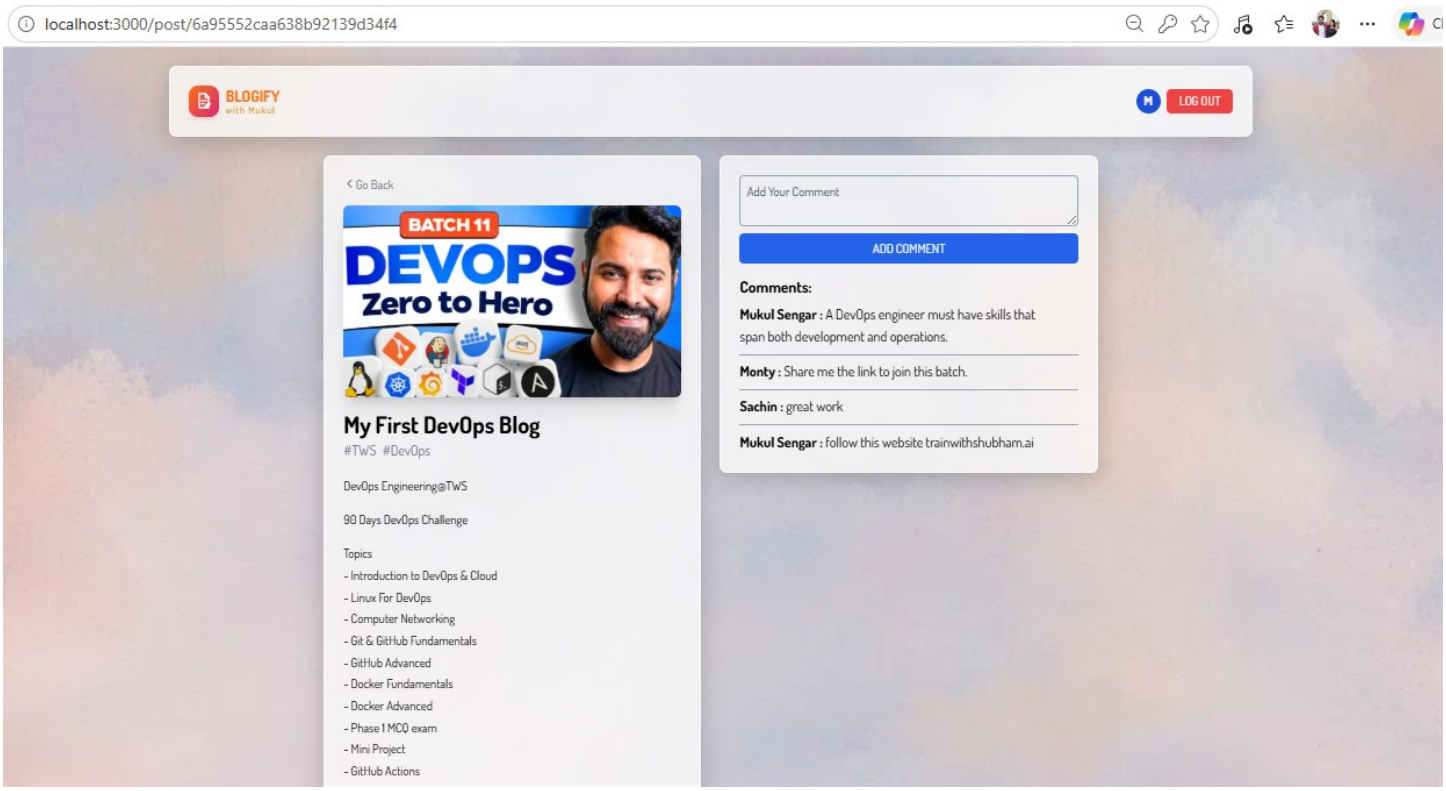
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
blog-backend:v3	344ce9db47e4	298MB	68.6MB	
blog-backend:v4	16a1fd9d5574	241MB	49.2MB	
blog-backendend:v1	4398a9c767eb	1.85GB	454MB	
blog-backendend:v2	ad7d5df08edf	317MB	74.7MB	
blog-frontend:v1	ed96b87cf37d	2.59GB	602MB	
blog-frontend:v2	78e21bbe3412	1.06GB	222MB	
blog-frontend:v3	9c3ce8178cc3	110MB	30.6MB	
blog-frontend:v4	dd63623728ec	37.3MB	10.1MB	
mongo:7.0	b6421fd6d1c5	1.18GB	299MB	
mongo:latest	5211c51171f5	1.15GB	303MB	

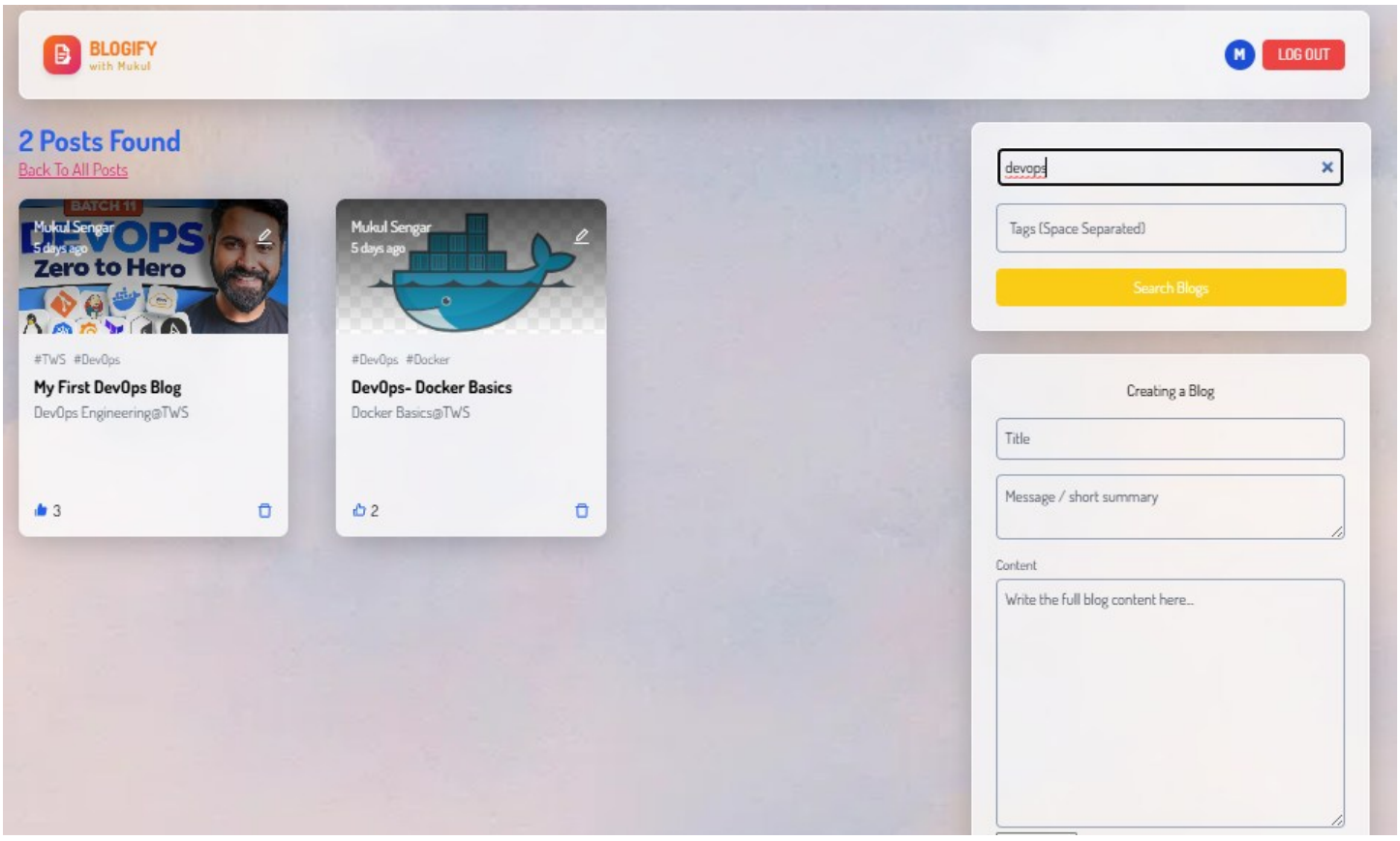
Image size has been reduces further to 241MB.
Backend v4: 298MB → 241MB (20% reduction)
Frontend v4: 110MB→37.3MB (60% reduction)

Step 4: Run docker containers for all tiers within same network.

1. Create Network:
   docker create network blog-network
2. Run database:
   docker run -d --name blog-db --network blog-network -v ./blog-db/db:/data/db/ -v ./blog-db/configdb:/data/configdb mongo
3. Run frontend: (http://localhost:3000)
   docker run -d --name blog-ui --network blog-network --env-file .env -p 3000:80 blog-frontend
4. Run Backend:
   docker run -d --name blog-bck --network blog-network --env-file .env blog-backend




localhost:3000/post/6a95552caa638b92139d34f4
BLOGIFY
with Mukul
M
LOG OUT
Go Back
BATCH 11
DEVOPS
Zero to Hero
My First DevOps Blog
#TWS #DevOps
DevOps Engineering@TWS
90 Days DevOps Challenge
Topics
- Introduction to DevOps & Cloud
- Linux For DevOps
- Computer Networking
- Git & GitHub Fundamentals
- GitHub Advanced
- Docker Fundamentals
- Docker Advanced
- Phase 1 MCQ exam
- Mini Project
- GitHub Actions
Add Your Comment
ADD COMMENT
Comments:
Mukul Sengar : A DevOps engineer must have skills that span both development and operations.
Monty : Share me the link to join this batch.
Sachin : great work
Mukul Sengar : follow this website trainwithshubham.ai


BLOGIFY
with Mukul
M
LOG OUT
2 Posts Found
Back To All Posts
Mukul Sengar
5 days ago
#TWS #DevOps
My First DevOps Blog
DevOps Engineering@TWS
3
Mukul Sengar
5 days ago
#DevOps #Docker
DevOps- Docker Basics
Docker Basics@TWS
2
devops
Tags (Space Separated)
Search Blogs
Creating a Blog
Title
Message / short summary
Content
Write the full blog content here...

Step 5: Write Docker Compose.

```
services:
  frontend:
    build:
      context: ./frontend
    container_name: blog-ui
    ports:
      - "3000:80"
    networks:
      - blog-network
    depends_on:
      - backend

  backend:
    build:
      context: ./backend
    container_name: blog-bck
    env_file:
      - ./backend/.env
    networks:
      - blog-network
    depends_on:
      - database

  database:
    image: mongo
    container_name: blog-db
    volumes:
      - ./blog-db/db:/data/db
      - ./blog-db/configdb:/data/configdb
    networks:
      - blog-network
    env_file:
      - ./.env

networks:
  blog-network:
```